maix.nn

maix.nn module

> You can use `maix.nn` to access this module with MaixPy
> This module is generated from MaixPy and MaixCDK

1. Module

module	brief
F	maix.nn.F module

2. Enum

2.1. SpeechDevice

speech device

item	describe
values	**DEVICE_NONE**: **DEVICE_PCM**: **DEVICE_MIC**: **DEVICE_WAV**:

> C++ defination code:

```
enum class SpeechDevice {
    DEVICE_NONE = -1,
    DEVICE_PCM,
    DEVICE_MIC,
    DEVICE_WAV,
}
```

2.2. SpeechDecoder

speech decoder type

item	describe
values	**DECODER_RAW**: **DECODER_DIG**: **DECODER_LVCSR**: **DECODER_KWS**: **DECODER_ALL**:

C++ defination code:

```
enum class SpeechDecoder {
    DECODER_RAW   = 1,
    DECODER_DIG   = 2,
    DECODER_LVCSR = 4,
    DECODER_KWS   = 8,
    DECODER_ALL   = 65535,
}
```

3. Variable

4. Function

5. Class

5.1. FaceDetector

FaceDetector class

> C++ defination code:
>
> ```
> class FaceDetector
> ```

5.1.1. __init__

```
def __init__(self, model: str = '', dual_buff: bool = True) -> None
```

Constructor of FaceDetector class

item	description
type	func
param	**model**: model path, default empty, you can load model later by load function. **dual_buff**: direction [in], prepare dual input output buffer to accelarate forward, that is, when NPU is forwarding we not wait and prepare the next input buff. If you want to ensure every time forward output the input's result, set this arg to false please. Default true to ensure speed.
throw	If model arg is not empty and load failed, will throw err::Exception.

item	description
static	False

> C++ defination code:
>
> ```
> FaceDetector(const string &model = "", bool dual_buff = true)
> ```

5.1.2. load

```
def load(self, model: str) -> maix.err.Err
```

Load model from file

item	description
type	func
param	**model**: Model path want to load
return	err::Err
static	False

> C++ defination code:
>
> ```
> err::Err load(const string &model)
> ```

5.1.3. detect

```
def detect(self, img: maix.image.Image, conf_th: float = 0.5, iou_th: float = 0.45, fit:
```

```
    maix.image.Fit = ...) -> list[...]
```

Detect objects from image

item	description
type	func
param	**img**: Image want to detect, if image's size not match model input's, will auto resize with fit method. **conf_th**: Confidence threshold, default 0.5. **iou_th**: IoU threshold, default 0.45. **fit**: Resize method, default image.Fit.FIT_CONTAIN.
throw	If image format not match model input format, will throw err::Exception.
return	Object list. In C++, you should delete it after use.
static	False

> C++ defination code:
>
> ```
> std::vector<nn::Object> *detect(image::Image &img, float conf_th = 0.5, float iou_th = 0.45, maix::image::Fit fit = maix::image::FIT_CONTAIN)
> ```

5.1.4. input_size

```
def input_size(self) -> maix.image.Size
```

Get model input size

item	description
type	func
return	model input size
static	False

> C++ defination code:
>
> ```
> image::Size input_size()
> ```

5.1.5. input_width

```
def input_width(self) -> int
```

Get model input width

item	description
type	func
return	model input size of width
static	False

> C++ defination code:
>
> ```
> int input_width()
> ```

5.1.6. input_height

```python
def input_height(self) -> int
```

Get model input height

item	description
type	func
return	model input size of height
static	False

> C++ defination code:
>
> ```cpp
> int input_height()
> ```

5.1.7. input_format

```python
def input_format(self) -> maix.image.Format
```

Get input image format

item	description
type	func
return	input image format, image::Format type.
static	False

> C++ defination code:
>
> ```cpp
> image::Format input_format()
> ```

5.1.8. mean

Get mean value, list type

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> mean
> ```

5.1.9. scale

Get scale value, list type

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> scale
> ```

5.2. OCR_Box

Object for OCR detect box

> C++ defination code:

```
class OCR_Box
```

5.2.1. __init__

```
def __init__(self, x1: int = 0, y1: int = 0, x2: int = 0, y2: int = 0, x3: int = 0, y3: int = 0, x4: int = 0, y4: int = 0) -> None
```

OCR_Box constructor

item	description
type	func
static	False

> C++ defination code:
>
> ```
> OCR_Box(int x1 = 0, int y1 = 0, int x2 = 0, int y2 = 0, int x3 = 0, int y3 = 0, int x4 = 0, int y4 = 0)
> ```

5.2.2. x1

left top point of box

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> int x1
> ```

5.2.3. y1

left top point of box

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> int y1
> ```

5.2.4. x2

right top point of box

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> int x2
> ```

5.2.5. y2

right top point of box

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> int y2
> ```

5.2.6. x3

right bottom point of box

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> int x3
> ```

5.2.7. y3

right bottom point of box

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> int y3
> ```

5.2.8. x4

left bottom point of box

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> int x4
> ```

5.2.9. y4

left bottom point of box

item	description
type	var
static	False

item	description
readonly	False

> C++ defination code:
>
> ```
> int y4
> ```

5.2.10. to_list

```
def to_list(self) -> list[int]
```

convert box point to a list type.

item	description
type	func
return	list type, element is int type, value [x1, y1, x2, y2, x3, y3, x4, y4].
static	False

> C++ defination code:
>
> ```
> std::vector<int> to_list()
> ```

5.3. OCR_Object

Object for OCR detect result

> C++ defination code:
>
> ```
> class OCR_Object
> ```

5.3.1. __init__

```
def __init__(self, box: OCR_Box, idx_list: list[int], char_list: list[str], score: float = 0, char_pos: list[int] = []) -> None
```

Constructor of Object for OCR detect result

item	description
type	func
param	**score**: score
static	False

> C++ defination code:
>
> ```
> OCR_Object(const nn::OCR_Box &box, const std::vector<int> &idx_list, const std::vector<std::string> &char_list, float score = 0, const std::vector<int> &char_pos = std::vector<int>())
> ```

5.3.2. box

OCR_Object box, 4 points box, first point at the left-top, clock-wise.

item	description
type	var
static	False

item	description
readonly	False

> C++ defination code:
>
> ```
> nn::OCR_Box box
> ```

5.3.3. score

Object score

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> float score
> ```

5.3.4. idx_list

chars' idx list, element is int type.

item	description
type	var
static	False
readonly	False

> C++ defination code:

```
std::vector<int> idx_list
```

5.3.5. char_pos

Chars' position relative to left

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<int> char_pos
> ```

5.3.6. char_str

```
def char_str(self) -> str
```

Get OCR_Object's charactors, return a string type.

item	description
type	func
return	All charactors in string type.
static	False

> C++ defination code:
>
> ```
> const std::string &char_str()
> ```

5.3.7. char_list

```python
def char_list(self) -> list[str]
```

Get OCR_Object's charactors, return a list type.

item	description
type	func
return	All charactors in list type.
static	False

> C++ defination code:
>
> ```cpp
> const std::vector<std::string> &char_list()
> ```

5.3.8. update_chars

```python
def update_chars(self, char_list: list[str]) -> None
```

Set OCR_Object's charactors

item	description
type	func
param	**char_list**: All charactors in list type.
static	False

> C++ defination code:

```
void update_chars(const std::vector<std::string> &char_list)
```

5.3.9. __str__

```
def __str__(self) -> str
```

OCR_Object info to string

item	description
type	func
return	OCR_Object info string
static	False

> C++ defination code:
>
> ```
> std::string to_str()
> ```

5.4. OCR_Objects

OCR_Objects Class for detect result

> C++ defination code:
>
> ```
> class OCR_Objects
> ```

5.4.1. __init__

```
def __init__(self) -> None
```

Constructor of OCR_Objects class

item	description
type	func
static	False

C++ defination code:

```
OCR_Objects()
```

5.4.2. add

```
def add(self, box: OCR_Box, idx_list: list[int], char_list: list[str], score: float = 0, char_pos: list[int] = []) -> OCR_Object
```

Add object to objects

item	description
type	func
throw	Throw exception if no memory
static	False

C++ defination code:

```
nn::OCR_Object &add(const nn::OCR_Box &box, const std::vector<int> &idx_list, const std::vector<std::string> &char_list, float score = 0, const std::vector<int> &char_pos
```

```
    = std::vector<int>())
```

5.4.3. remove

```
def remove(self, idx: int) -> maix.err.Err
```

Remove object form objects

item	description
type	func
static	False

> C++ defination code:
>
> ```
> err::Err remove(int idx)
> ```

5.4.4. at

```
def at(self, idx: int) -> OCR_Object
```

Get object item

item	description
type	func
static	False

> C++ defination code:
>
> ```
> nn::OCR_Object &at(int idx)
> ```

5.4.5. __getitem__

```python
def __getitem__(self, idx: int) -> OCR_Object
```

Get object item

item	description
type	func
static	False

> C++ defination code:
>
> ```cpp
> nn::OCR_Object &operator[](int idx)
> ```

5.4.6. __len__

```python
def __len__(self) -> int
```

Get size

item	description
type	func
static	False

> C++ defination code:
>
> ```cpp
> size_t size()
> ```

5.4.7. __iter__

```
def __iter__(self) -> typing.Iterator
```

Begin

item	description
type	func
static	False

> C++ defination code:
>
> ```
> std::vector<OCR_Object*>::iterator begin()
> ```

5.5. SelfLearnClassifier

SelfLearnClassifier

> C++ defination code:
>
> ```
> class SelfLearnClassifier
> ```

5.5.1. __init__

```
def __init__(self, model: str = '', dual_buff: bool = True) -> None
```

Construct a new SelfLearnClassifier object

item	description
type	func

item	description
param	**model**: MUD model path, if empty, will not load model, you can call load_model() later. if not empty, will load model and will raise err::Exception if load failed. **dual_buff**: direction [in], prepare dual input output buffer to accelarate forward, that is, when NPU is forwarding we not wait and prepare the next input buff. If you want to ensure every time forward output the input's result, set this arg to false please. Default true to ensure speed.
static	False

> C++ defination code:
>
> ```
> SelfLearnClassifier(const std::string &model = "", bool dual_buff = true)
> ```

5.5.2. load_model

```
def load_model(self, model: str) -> maix.err.Err
```

Load model from file, model format is .mud,\nMUD file should contain [extra] section, have key-values:\n- model_type: classifier_no_top\n- input_type: rgb or bgr\n- mean: 123.675, 116.28, 103.53\n- scale: 0.017124753831663668, 0.01750700280112045, 0.017429193899782137

item	description
type	func
param	**model**: MUD model path
return	error code, if load failed, return error code
static	False

> C++ defination code:
>
> ```
> err::Err load_model(const string &model)
> ```

5.5.3. classify

```
def classify(self, img: maix.image.Image, fit: maix.image.Fit = ...) -> list[tuple[int, float]]
```

Classify image

item	description
type	func
param	**img**: image, format should match model input_type， or will raise err.Exception **fit**: image resize fit mode, default Fit.FIT_COVER, see image.Fit.

item	description
throw	If error occurred, will raise err::Exception, you can find reason in log, mostly caused by args error or hardware error.
return	result, a list of (idx, distance), smaller distance means more similar. In C++, you need to delete it after use.
static	False

> C++ defination code:
>
> ```
> std::vector<std::pair<int, float>> *classify(image::Image &img, image::Fit fit = image::FIT_COVER)
> ```

5.5.4. add_class

```
def add_class(self, img: maix.image.Image, fit: maix.image.Fit = ...) -> None
```

Add a class to recognize

item	description
type	func
param	**img**: Add a image as a new class **fit**: image resize fit mode, default Fit.FIT_COVER, see image.Fit.

item	description
static	False

C++ defination code:

```
void add_class(image::Image &img, image::Fit fit = image::FIT_COVER)
```

5.5.5. class_num

```
def class_num(self) -> int
```

Get class number

item	description
type	func
static	False

C++ defination code:

```
int class_num()
```

5.5.6. rm_class

```
def rm_class(self, idx: int) -> maix.err.Err
```

Remove a class

item	description
type	func
param	**idx**: index, value from 0 to class_num();
static	False

> C++ defination code:
>
> ```
> err::Err rm_class(int idx)
> ```

5.5.7. add_sample

```
def add_sample(self, img: maix.image.Image, fit: maix.image.Fit = ...) -> None
```

Add sample, you should call learn method after add some samples to learn classes.\nSample image can be any of classes we already added.

item	description
type	func
param	**img**: Add a image as a new sample.
static	False

> C++ defination code:
>
> ```
> void add_sample(image::Image &img, image::Fit fit = image::FIT_COVER)
> ```

5.5.8. rm_sample

```
def rm_sample(self, idx: int) -> maix.err.Err
```

Remove a sample

item	description
type	func
param	**idx**: index, value from 0 to sample_num();
static	False

> C++ defination code:
>
> ```
> err::Err rm_sample(int idx)
> ```

5.5.9. sample_num

```
def sample_num(self) -> int
```

Get sample number

item	description
type	func
static	False

> C++ defination code:
>
> ```
> int sample_num()
> ```

5.5.10. learn

```
def learn(self) -> int
```

Start auto learn class features from classes image and samples.\nYou should call this method after you add some samples.

item	description
type	func
return	learn epoch(times), 0 means learn nothing.
static	False

> C++ defination code:
>
> ```
> int learn()
> ```

5.5.11. clear

```
def clear(self) -> None
```

Clear all class and samples

item	description
type	func
static	False

> C++ defination code:
>
> ```
> void clear()
> ```

5.5.12. input_size

```
def input_size(self) ->
maix.image.Size
```

Get model input size, only for image input

item	description
type	func
return	model input size
static	False

> C++ defination code:
>
> ```
> image::Size input_size()
> ```

5.5.13. input_width

```
def input_width(self) -> int
```

Get model input width, only for image input

item	description
type	func
return	model input size of width
static	False

> C++ defination code:

```
int input_width()
```

5.5.14. input_height

```
def input_height(self) -> int
```

Get model input height, only for image input

item	description
type	func
return	model input size of height
static	False

> C++ defination code:
>
> ```
> int input_height()
> ```

5.5.15. input_format

```
def input_format(self) ->
maix.image.Format
```

Get input image format, only for image input

item	description
type	func
return	input image format, image::Format type.
static	False

> C++ defination code:
>
> ```
> image::Format input_format()
> ```

5.5.16. input_shape

```
def input_shape(self) -> list[int]
```

Get input shape, if have multiple input, only return first input shape

item	description
type	func
return	input shape, list type
static	False

> C++ defination code:
>
> ```
> std::vector<int> input_shape()
> ```

5.5.17. save

```
def save(self, path: str, labels: list[str] = []) -> maix.err.Err
```

Save features and labels to a binary file

item	description
type	func

item	description
param	**path**: file path to save, e.g. /root/my_classes.bin **labels**: class labels, can be None, or length must equal to class num, or will return err::Err
return	maix.err.Err if labels exists but length not equal to class num, or save file failed, or class num is 0.
static	False

> C++ defination code:
>
> ```
> err::Err save(const std::string &path, const std::vector<std::string> &labels = std::vector<std::string>())
> ```

5.5.18. load

```
def load(self, path: str) -> list[str]
```

Load features info from binary file

item	description
type	func
param	**path**: feature info binary file path, e.g. /root/my_classes.bin
static	False

C++ defination code:

```
std::vector<std::string> load(const std::string &path)
```

5.5.19. labels

Labels list

item	description
type	var
static	False
readonly	False

C++ defination code:

```
std::vector<string> labels
```

5.5.20. label_path

Label file path

item	description
type	var
static	False
readonly	False

C++ defination code:

```
std::string label_path
```

5.5.21. mean

Get mean value, list type

item	description
type	var
static	False
readonly	False

C++ defination code:

```
std::vector<float> mean
```

5.5.22. scale

Get scale value, list type

item	description
type	var
static	False
readonly	False

C++ defination code:

```
std::vector<float> scale
```

5.6. FaceLandmarksObject

FaceLandmarksObject class

> C++ defination code:
>
> ```
> class FaceLandmarksObject
> ```

5.6.1. __init__

```
def __init__(self) -> None
```

Valid or not(score > conf_th when detect).

item	description
type	func
static	False

> C++ defination code:
>
> ```
> FaceLandmarksObject()
> ```

5.6.2. valid

Valid or not(score > conf_th when detect).

item	description
type	var
static	False
readonly	False

> C++ defination code:

```
bool valid
```

5.6.3. score

whether face in image score, value from 0 to 1.0.

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> float score
> ```

5.6.4. points

landmarks points, format: x0, y0, ..., xn-1, yn-1.

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<int> points
> ```

5.6.5. points_z

landmarks points, format: z0, z1, ..., zn-1.

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<int> points_z
> ```

5.7. FaceLandmarks

FaceLandmarks class

> C++ defination code:
>
> ```
> class FaceLandmarks
> ```

5.7.1. __init__

```
def __init__(self, model: str = '') -> None
```

Constructor of FaceLandmarks class

item	description
type	func
param	**model**: model path, default empty, you can load model later by load

item	description
	function.
throw	If model arg is not empty and load failed, will throw err::Exception.
static	False

> C++ defination code:
>
> ```
> FaceLandmarks(const string &model = "")
> ```

5.7.2. load

```
def load(self, model: str) -> maix.err.Err
```

Load model from file

item	description
type	func
param	**model**: Model path want to load
return	err::Err
static	False

> C++ defination code:
>
> ```
> err::Err load(const string &model)
> ```

5.7.3. detect

```
def detect(self, img: maix.image.Image, conf_th: float = 0.5, landmarks_abs: bool = True, landmarks_rel: bool = False) -> FaceLandmarksObject
```

Detect objects from image

item	description
type	func
param	**img**: Image want to detect, if image's si model input's, will auto resize with fit m **conf_th**: Hand detect confidence thresh **landmarks_rel**: outputs the relative coo points with respect to the top-left verte In obj.points, the last 21x2 values are ar x0y0x1y1...x20y20. Value from 0 to obj.w.
throw	If image format not match model input throw err::Exception.
return	Object list. In C++, you should delete it Object's points value format: box_toplef box_topleft_y, box_topright_x, box_topri box_bottomright_x, box_bottomright_y, box_bottomleft_x, box_bottomleft_y, x0, y0, z1, x1, y1, z2, ..., x20, y20, z20. If landmarks_rel is True, will be box_topl box_topleft_y...,x20,y20,z20,x0_rel,y0_rel,. Z is depth, the larger the value, the fartl the palm, and the positive value means camera.

item	description
static	False

> C++ defination code:
>
> ```
> nn::FaceLandmarksObject *detect(image::Image &img, float conf_th = 0.5, bool landmarks_abs = true, bool landmarks_rel = false)
> ```

5.7.4. crop_image

```
def crop_image(self, img: maix.image.Image, x: int, y: int, w: int, h: int, points: list[int], new_width: int = -1, new_height: int = -1, scale: float = 1.2) -> maix.image.Image
```

Crop image from source image by 2 points(2 eyes)

item	description
type	func

item	description
param	**x,y,w,h**: face rectangle, x,y is left-top point. **img**: source image **points**: 2 points, eye_left_x, eye_left_y, eye_right_x, eye_right_y **scale**: crop size scale relative to rectangle's max side length(w or h), final value is `scale *max(w, h)` ,default 1.2.
static	False

> C++ defination code:
>
> ```
> maix::image::Image *crop_image(maix::image::Image &img, int x, int y, int w, int h, std::vector<int> points, int new_width = -1, int new_height = -1, float scale = 1.2)
> ```

5.7.5. input_size

```
def input_size(self, detect: bool = True) -> maix.image.Size
```

Get model input size

item	description
type	func

item	description
param	**detect**: detect or landmarks model, default true.
return	model input size
static	False

> C++ defination code:
>
> ```
> image::Size input_size(bool detect = true)
> ```

5.7.6. input_width

```
def input_width(self, detect: bool = True) -> int
```

Get model input width

item	description
type	func
param	**detect**: detect or landmarks model, default true.
return	model input size of width
static	False

> C++ defination code:
>
> ```
> int input_width(bool detect = true)
> ```

5.7.7. input_height

```
def input_height(self, detect: bool = True) -> int
```

Get model input height

item	description
type	func
param	**detect**: detect or landmarks model, default true.
return	model input size of height
static	False

> C++ defination code:
>
> ```
> int input_height(bool detect = true)
> ```

5.7.8. input_format

```
def input_format(self) -> maix.image.Format
```

Get input image format

item	description
type	func
return	input image format, image::Format type.

item	description
static	False

> C++ defination code:
>
> ```
> image::Format input_format()
> ```

5.7.9. draw_face

```
def draw_face(self, img: maix.image.Image, points: list[int], num: int, points_z: list[int] = [], r_min: int = 2, r_max: int = 4) -> None
```

Draw hand and landmarks on image

item	description
type	func
param	**img**: image object, maix.image.Image type. **leftright,**: 0 means left, 1 means right **points**: points result from detect method: x0, y0, x1, y1, ..., xn-1, yn-1. **points_z**: points result from detect method: z0, z1, ..., zn-1. **r_min**: min radius of points. **r_max**: min radius of points.
static	False

> C++ defination code:

```
void draw_face(image::Image &img, const std::vector<int> &points, int num, const std::vector<int> &points_z=std::vector<int>(), int r_min = 2, int r_max = 4)
```

5.7.10. mean

Get mean value, list type

item	description
type	var
static	False
readonly	False

C++ defination code:

```
std::vector<float> mean
```

5.7.11. scale

Get scale value, list type

item	description
type	var
static	False
readonly	False

C++ defination code:

```
std::vector<float> scale
```

5.7.12. landmarks_num

landmarks number.

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> int landmarks_num
> ```

5.8. FaceObject

Face object

> C++ defination code:
>
> ```
> class FaceObject
> ```

5.8.1. __init__

```
def __init__(self, x: int = 0, y: int = 0, w: int = 0, h: int = 0, class_id: int = 0, score: float = 0, points: list[int] = [], feature: list[float] = [], face: maix.image.Image = ...) -> None
```

Constructor

item	description
type	func
static	False

> C++ defination code:
>
> ```
> FaceObject(int x = 0, int y = 0, int w = 0, int h = 0, int class_id = 0, float score = 0, std::vector<int> points = std::vector<int>(), std::vector<float> feature = std::vector<float>(), image::Image face = image::Image())
> ```

5.8.2. __str__

```
def __str__(self) -> str
```

FaceObject info to string

item	description
type	func
return	FaceObject info string
static	False

> C++ defination code:
>
> ```
> std::string to_str()
> ```

5.8.3. x

FaceObject left top coordinate x

item	description
type	var
static	False
readonly	False

C++ defination code:

```
int x
```

5.8.4. y

FaceObject left top coordinate y

item	description
type	var
static	False
readonly	False

C++ defination code:

```
int y
```

5.8.5. w

FaceObject width

item	description
type	var

item	description
static	False
readonly	False

C++ defination code:

```
int w
```

5.8.6. h

FaceObject height

item	description
type	var
static	False
readonly	False

C++ defination code:

```
int h
```

5.8.7. class_id

FaceObject class id

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> int class_id
> ```

5.8.8. score

FaceObject score

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> float score
> ```

5.8.9. points

keypoints

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<int> points
> ```

5.8.10. feature

feature, float list type

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> feature
> ```

5.8.11. face

face image

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> image::Image face
> ```

5.9. FaceObjects

Objects Class for detect result

> C++ defination code:

```
class FaceObjects
```

5.9.1. __init__

```
def __init__(self) -> None
```

Constructor of FaceObjects class

item	description
type	func
static	False

> C++ defination code:
>
> ```
> FaceObjects()
> ```

5.9.2. add

```
def add(self, x: int = 0, y: int = 0, w: int = 0, h: int = 0, class_id: int = 0, score: float = 0, points: list[int] = [], feature: list[float] = [], face: maix.image.Image = ...) -> FaceObject
```

Add object to FaceObjects

item	description
type	func
throw	Throw exception if no memory

item	description
static	False

> C++ defination code:
>
> ```
> nn::FaceObject &add(int x = 0, int y = 0, int w = 0, int h = 0, int class_id = 0, float score = 0, std::vector<int> points = std::vector<int>(), std::vector<float> feature = std::vector<float>(), image::Image face = image::Image())
> ```

5.9.3. remove

```
def remove(self, idx: int) -> maix.err.Err
```

Remove object form FaceObjects

item	description
type	func
static	False

> C++ defination code:
>
> ```
> err::Err remove(int idx)
> ```

5.9.4. at

```
def at(self, idx: int) -> FaceObject
```

Get object item

item	description
type	func
static	False

> C++ defination code:
>
> ```
> nn::FaceObject &at(int idx)
> ```

5.9.5. __getitem__

```
def __getitem__(self, idx: int) ->
FaceObject
```

Get object item

item	description
type	func
static	False

> C++ defination code:
>
> ```
> nn::FaceObject &operator[](int
> idx)
> ```

5.9.6. __len__

```
def __len__(self) -> int
```

Get size

item	description
type	func
static	False

> C++ defination code:
>
> ```
> size_t size()
> ```

5.9.7. __iter__

```
def __iter__(self) -> typing.Iterator
```

Begin

item	description
type	func
static	False

> C++ defination code:
>
> ```
> std::vector<FaceObject*>::iterator begin()
> ```

5.10. FaceRecognizer

FaceRecognizer class

> C++ defination code:

```
class FaceRecognizer
```

5.10.1. __init__

```
def __init__(self, detect_model: str = '', feature_model: str = '', dual_buff: bool = True) -> None
```

Constructor of FaceRecognizer class

item	description
type	func
param	**detect_model**: face detect model path, default empty, you can load model later by load function. **feature_model**: feature extract model **dual_buff**: direction [in], prepare dual input output buffer to accelarate forward, that is, when NPU is forwarding we not wait and prepare the next input buff. If you want to ensure every time forward output the input's result, set this arg to false please. Default true to ensure speed.
throw	If model arg is not empty and load failed, will throw err::Exception.
static	False

> C++ defination code:

```
FaceRecognizer(const string &detect_model = "", const string &feature_model = "", bool dual_buff = true)
```

5.10.2. load

```
def load(self, detect_model: str, feature_model: str) -> maix.err.Err
```

Load model from file

item	description
type	func
param	**detect_model**: face detect model path, default empty, you can load model later by load function. **feature_model**: feature extract model
return	err::Err
static	False

> C++ defination code:
>
> ```
> err::Err load(const string &detect_model, const string &feature_model)
> ```

5.10.3. recognize

```
def recognize(self, img: maix.image.Image, conf_th: float = 0.5, iou_th: float = 0.45,
```

```
    compare_th: float = 0.8,
    get_feature: bool = False, get_face:
    bool = False, fit: maix.image.Fit =
    ...) -> FaceObjects
```

Detect objects from image

item	description
type	func
param	**img**: Image want to detect, if image's size not match model input's, will auto resize with fit method. **conf_th**: Detect confidence threshold, default 0.5. **iou_th**: Detect IoU threshold, default 0.45. **compare_th**: Compare two face score threshold, default 0.8, if two faces' score < this value, will see this face fas unknown. **get_feature**: return feature or not, if true will copy features to result, if false will not copy feature to result to save time and memory. **get_face**: return face image or not, if true result object's face attribute will valid, or face sttribute is empty. Get face image will alloc memory and copy image, so will lead to slower speed. **fit**: Resize method, default image.Fit.FIT_CONTAIN.

item	description
throw	If image format not match model input format, will throw err::Exception.
return	FaceObjects object. In C++, you should delete it after use.
static	False

> C++ defination code:
>
> ```
> nn::FaceObjects *recognize(image::Image &img, float conf_th = 0.5, float iou_th = 0.45, float compare_th = 0.8, bool get_feature = false, bool get_face = false, maix::image::Fit fit = maix::image::FIT_CONTAIN)
> ```

5.10.4. add_face

```
def add_face(self, face: FaceObject, label: str) -> maix.err.Err
```

Add face to lib

item	description
type	func
param	**face**: face object, find by recognize **label**: face label(name)
static	False

> C++ defination code:
>
> ```
> err::Err add_face(nn::FaceObject *face, const std::string &label)
> ```

5.10.5. remove_face

```
def remove_face(self, idx: int = -1, label: str = '') -> maix.err.Err
```

remove face from lib

item	description
type	func
param	**idx**: index of face in lib, default -1 means use label, value [0,face_num), idx and label must have one, idx have high priotiry. **label**: which face to remove, default to empty string mean use idx, idx and label must have one, idx have high priotiry.
static	False

> C++ defination code:
>
> ```
> err::Err remove_face(int idx = -1, const std::string &label = "")
> ```

5.10.6. save_faces

```
def save_faces(self, path: str) -> maix.err.Err
```

Save faces info to a file

item	description
type	func
param	**path**: where to save, string type.
return	err.Err type
static	False

> C++ defination code:
>
> ```
> err::Err save_faces(const std::string &path)
> ```

5.10.7. load_faces

```
def load_faces(self, path: str) -> maix.err.Err
```

Load faces info from a file

item	description
type	func
param	**path**: from where to load, string type.
return	err::Err type
static	False

> C++ defination code:

```
err::Err load_faces(const
std::string &path)
```

5.10.8. input_size

```
def input_size(self) ->
maix.image.Size
```

Get model input size

item	description
type	func
return	model input size
static	False

> C++ defination code:
>
> ```
> image::Size input_size()
> ```

5.10.9. input_width

```
def input_width(self) -> int
```

Get model input width

item	description
type	func
return	model input size of width
static	False

> C++ defination code:
>
> ```
> int input_width()
> ```

5.10.10. input_height

```
def input_height(self) -> int
```

Get model input height

item	description
type	func
return	model input size of height
static	False

> C++ defination code:
>
> ```
> int input_height()
> ```

5.10.11. input_format

```
def input_format(self) ->
maix.image.Format
```

Get input image format

item	description
type	func

item	description
return	input image format, image::Format type.
static	False

> C++ defination code:
>
> ```
> image::Format input_format()
> ```

5.10.12. mean_detector

Get mean value, list type

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> mean_detector
> ```

5.10.13. scale_detector

Get scale value, list type

item	description
type	var
static	False

item	description
readonly	False

C++ defination code:

```
std::vector<float> scale_detector
```

5.10.14. mean_feature

Get mean value, list type

item	description
type	var
static	False
readonly	False

C++ defination code:

```
std::vector<float> mean_feature
```

5.10.15. scale_feature

Get scale value, list type

item	description
type	var
static	False
readonly	False

C++ defination code:

```
std::vector<float> scale_feature
```

5.10.16. labels

labels, list type, first is "unknown"

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<std::string> labels
> ```

5.10.17. features

features

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<std::vector<float>>
> features
> ```

5.11. YOLOv8

YOLOv8 class

C++ defination code:

```
class YOLOv8
```

5.11.1. __init__

```
def __init__(self, model: str = '', dual_buff: bool = True) -> None
```

Constructor of YOLOv8 class

item	description
type	func
param	**model**: model path, default empty, you can load model later by load function. **dual_buff**: direction [in], prepare dual input output buffer to accelarate forward, that is, when NPU is forwarding we not wait and prepare the next input buff. If you want to ensure every time forward output the input's result, set this arg to false please. Default true to ensure speed.
throw	If model arg is not empty and load failed, will throw err::Exception.
static	False

> C++ defination code:
>
> ```
> YOLOv8(const string &model = "", bool dual_buff = true)
> ```

5.11.2. load

```
def load(self, model: str) -> maix.err.Err
```

Load model from file

item	description
type	func
param	**model**: Model path want to load
return	err::Err
static	False

> C++ defination code:
>
> ```
> err::Err load(const string &model)
> ```

5.11.3. detect

```
def detect(self, img: maix.image.Image, conf_th: float = 0.5, iou_th: float = 0.45, fit: maix.image.Fit = ..., keypoint_th: float = 0.5, sort: int = 0) -> ...
```

Detect objects from image

item	description
type	func
param	**img**: Image want to detect, if image's size not match model input's, will auto resize with fit method. **conf_th**: Confidence threshold, default 0.5. **iou_th**: IoU threshold, default 0.45. **fit**: Resize method, default image.Fit.FIT_CONTAIN. **keypoint_th**: keypoint threshold, default 0.5, only for yolov8-pose model. **sort**: sort result according to object size, default 0 means not sort, 1 means bigger in front, -1 means smaller in front.
throw	If image format not match model input format, will throw err::Exception.
return	Object list. In C++, you should delete it after use. If model is yolov8-pose, object's points have value, and if points' value < 0 means that point is invalid(conf < keypoint_th).
static	False

> C++ defination code:

```
nn::Objects *detect(image::Image &img, float conf_th = 0.5, float iou_th = 0.45, maix::image::Fit fit = maix::image::FIT_CONTAIN, float keypoint_th = 0.5, int sort = 0)
```

5.11.4. input_size

```
def input_size(self) -> maix.image.Size
```

Get model input size

item	description
type	func
return	model input size
static	False

> C++ defination code:
>
> ```
> image::Size input_size()
> ```

5.11.5. input_width

```
def input_width(self) -> int
```

Get model input width

item	description
type	func

item	description
return	model input size of width
static	False

> C++ defination code:
>
> ```
> int input_width()
> ```

5.11.6. input_height

```
def input_height(self) -> int
```

Get model input height

item	description
type	func
return	model input size of height
static	False

> C++ defination code:
>
> ```
> int input_height()
> ```

5.11.7. input_format

```
def input_format(self) ->
maix.image.Format
```

Get input image format

item	description
type	func
return	input image format, image::Format type.
static	False

> C++ defination code:
>
> ```
> image::Format input_format()
> ```

5.11.8. draw_pose

```
def draw_pose(self, img: maix.image.Image, points: list[int], radius: int = 4, color: maix.image.Color = ..., colors: list[maix.image.Color] = [], body: bool = True, close: bool = False) -> None
```

Draw pose keypoints on image

item	description
type	func
param	**img**: image object, maix.image.Image type. **points**: keypoits, int list type, [x, y, x, y ...] **radius**: radius of points. **color**: color of points. **colors**: assign colors for points, list

item	description
	type, element is image.Color object. **body**: true, if points' length is 17*2 and body is ture, will draw lines as human body, if set to false won't draw lines, default true.* *close*: connect all points to close a polygon, default false.
static	False

> C++ defination code:
>
> ```
> void draw_pose(image::Image &img, std::vector<int> points, int radius = 4, image::Color color = image::COLOR_RED, const std::vector<image::Color> &colors = std::vector<image::Color>(), bool body = true, bool close = false)
> ```

5.11.9. draw_seg_mask

```
def draw_seg_mask(self, img: maix.image.Image, x: int, y: int, seg_mask: maix.image.Image, threshold: int = 127) -> None
```

Draw segmentation on image

item	description
type	func

item	description
param	**img**: image object, maix.image.Image type. **seg_mask**: segmentation mask image by detect method, a grayscale image **threshold**: only mask's value > threshold will be draw on image, value from 0 to 255.
static	False

> C++ defination code:
>
> ```
> void draw_seg_mask(image::Image &img, int x, int y, image::Image &seg_mask, int threshold = 127)
> ```

5.11.10. labels

Labels list

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<string> labels
> ```

5.11.11. label_path

Label file path

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::string label_path
> ```

5.11.12. mean

Get mean value, list type

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> mean
> ```

5.11.13. scale

Get scale value, list type

item	description
type	var

item	description
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> scale
> ```

5.12. NanoTrack

NanoTrack class

> C++ defination code:
>
> ```
> class NanoTrack
> ```

5.12.1. __init__

```
def __init__(self, model: str = '') -> None
```

Constructor of NanoTrack class

item	description
type	func
param	**model**: model path, default empty, you can load model later by load function.
throw	If model arg is not empty and load failed, will throw err::Exception.

item	description
static	False

> C++ defination code:
>
> ```
> NanoTrack(const string &model = "")
> ```

5.12.2. load

```
def load(self, model: str) -> maix.err.Err
```

Load model from file

item	description
type	func
param	**model**: Model path want to load
return	err::Err
static	False

> C++ defination code:
>
> ```
> err::Err load(const string &model)
> ```

5.12.3. init

```
def init(self, img: maix.image.Image, x: int, y: int, w:
```

```
int, h: int) -> None
```

Init tracker, give tacker first target image and target position.

item	description
type	func
param	**img**: Image want to detect, target should be in this image. **x**: the target position left top coordinate x. **y**: the target position left top coordinate y. **w**: the target width. **h**: the target height.
throw	If image format not match model input format, will throw err::Exception.
static	False

> C++ defination code:
>
> ```
> void init(image::Image &img, int x, int y, int w, int h)
> ```

5.12.4. track

```
def track(self, img: maix.image.Image, threshold: float = 0.9) -> ...
```

Track object acoording to last object position and the init function learned target feature.

item	description
type	func
param	**img**: image to detect object and track, can be any resolution, before detect it will crop a area according to last time target's position. **threshold**: If score < threshold, will see this new detection is invalid, but remain return this new detecion, default 0.9.
return	object, position and score, and detect area in points's first 4 element(x, y, w, h, center_x, center_y, input_size, target_size)
static	False

> C++ defination code:
>
> ```
> nn::Object track(image::Image &img, float threshold = 0.9)
> ```

5.12.5. input_size

```
def input_size(self) -> maix.image.Size
```

Get model input size

item	description
type	func
return	model input size
static	False

> C++ defination code:
>
> ```
> image::Size input_size()
> ```

5.12.6. input_width

```
def input_width(self) -> int
```

Get model input width

item	description
type	func
return	model input size of width
static	False

> C++ defination code:
>
> ```
> int input_width()
> ```

5.12.7. input_height

```
def input_height(self) -> int
```

Get model input height

item	description
type	func
return	model input size of height
static	False

> C++ defination code:
>
> ```
> int input_height()
> ```

5.12.8. input_format

```
def input_format(self) ->
maix.image.Format
```

Get input image format

item	description
type	func
return	input image format, image::Format type.
static	False

> C++ defination code:
>
> ```
> image::Format input_format()
> ```

5.12.9. mean

Get mean value, list type

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> mean
> ```

5.12.10. scale

Get scale value, list type

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> scale
> ```

5.13. YOLO11

YOLO11 class

> C++ defination code:
>
> ```
> class YOLO11
> ```

5.13.1. __init__

```python
def __init__(self, model: str = '', dual_buff: bool = True) -> None
```

Constructor of YOLO11 class

item	description
type	func
param	**model**: model path, default empty, you can load model later by load function. **dual_buff**: direction [in], prepare dual input output buffer to accelarate forward, that is, when NPU is forwarding we not wait and prepare the next input buff. If you want to ensure every time forward output the input's result, set this arg to false please. Default true to ensure speed.
throw	If model arg is not empty and load failed, will throw err::Exception.
static	False

> C++ defination code:
>
> ```cpp
> YOLO11(const string &model = "", bool dual_buff = true)
> ```

5.13.2. load

```python
def load(self, model: str) -> maix.err.Err
```

Load model from file

item	description
type	func
param	**model**: Model path want to load
return	err::Err
static	False

> C++ defination code:
>
> ```cpp
> err::Err load(const string &model)
> ```

5.13.3. detect

```python
def detect(self, img: maix.image.Image, conf_th: float = 0.5, iou_th: float = 0.45, fit: maix.image.Fit = ..., keypoint_th: float = 0.5, sort: int = 0) -> ...
```

Detect objects from image

item	description
type	func
param	**img**: Image want to detect, if image's size not match model input's, will auto resize with fit method.

item	description
	conf_th: Confidence threshold, default 0.5. **iou_th**: IoU threshold, default 0.45. **fit**: Resize method, default image.Fit.FIT_CONTAIN. **keypoint_th**: keypoint threshold, default 0.5, only for yolo11-pose model. **sort**: sort result according to object size, default 0 means not sort, 1 means bigger in front, -1 means smaller in front.
throw	If image format not match model input format, will throw err::Exception.
return	Object list. In C++, you should delete it after use. If model is yolo11-pose, object's points have value, and if points' value < 0 means that point is invalid(conf < keypoint_th).
static	False

> C++ defination code:
>
> ```
> nn::Objects *detect(image::Image &img, float conf_th = 0.5, float iou_th = 0.45, maix::image::Fit fit = maix::image::FIT_CONTAIN, float keypoint_th = 0.5, int sort
> ```

```
= 0)
```

5.13.4. input_size

```
def input_size(self) -> maix.image.Size
```

Get model input size

item	description
type	func
return	model input size
static	False

> C++ defination code:
>
> ```
> image::Size input_size()
> ```

5.13.5. input_width

```
def input_width(self) -> int
```

Get model input width

item	description
type	func
return	model input size of width
static	False

> C++ defination code:

```
int input_width()
```

5.13.6. input_height

```
def input_height(self) -> int
```

Get model input height

item	description
type	func
return	model input size of height
static	False

> C++ defination code:
>
> ```
> int input_height()
> ```

5.13.7. input_format

```
def input_format(self) ->
maix.image.Format
```

Get input image format

item	description
type	func
return	input image format, image::Format type.
static	False

> C++ defination code:
>
> ```
> image::Format input_format()
> ```

5.13.8. draw_pose

```
def draw_pose(self, img: maix.image.Image, points: list[int], radius: int = 4, color: maix.image.Color = ..., colors: list[maix.image.Color] = [], body: bool = True, close: bool = False) -> None
```

Draw pose keypoints on image

item	description
type	func
param	**img**: image object, maix.image.Image type. **points**: keypoits, int list type, [x, y, x, y ...] **radius**: radius of points. **color**: color of points. **colors**: assign colors for points, list type, element is image.Color object. **body**: true, if points' length is 17_2 and body is ture, will draw lines as human body, if set to false won't draw lines, default true. close_*: connect all points to close a polygon, default false.

item	description
static	False

C++ defination code:

```
void draw_pose(image::Image &img, std::vector<int> points, int radius = 4, image::Color color = image::COLOR_RED, const std::vector<image::Color> &colors = std::vector<image::Color>(), bool body = true, bool close = false)
```

5.13.9. draw_seg_mask

```
def draw_seg_mask(self, img: maix.image.Image, x: int, y: int, seg_mask: maix.image.Image, threshold: int = 127) -> None
```

Draw segmentation on image

item	description
type	func
param	**img**: image object, maix.image.Image type. **seg_mask**: segmentation mask image by detect method, a grayscale image **threshold**: only mask's value > threshold will be draw on image, value from 0 to 255.
static	False

C++ defination code:

```
void draw_seg_mask(image::Image &img, int x, int y, image::Image &seg_mask, int threshold = 127)
```

5.13.10. labels

Labels list

item	description
type	var
static	False
readonly	False

C++ defination code:

```
std::vector<string> labels
```

5.13.11. label_path

Label file path

item	description
type	var
static	False
readonly	False

C++ defination code:

```
std::string label_path
```

5.13.12. mean

Get mean value, list type

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> mean
> ```

5.13.13. scale

Get scale value, list type

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> scale
> ```

5.14. Retinaface

Retinaface class

> C++ defination code:
>
> ```
> class Retinaface
> ```

5.14.1. __init__

```
def __init__(self, model: str = '', dual_buff: bool = True) -> None
```

Constructor of Retinaface class

item	description
type	func
param	**model**: model path, default empty, you can load model later by load function. **dual_buff**: direction [in], prepare dual input output buffer to accelarate forward, that is, when NPU is forwarding we not wait and prepare the next input buff. If you want to ensure every time forward output the input's result, set this arg to false please. Default true to ensure speed.
throw	If model arg is not empty and load failed, will throw err::Exception.
static	False

> C++ defination code:
>
> ```
> Retinaface(const string &model = "", bool dual_buff = true)
> ```

5.14.2. load

```
def load(self, model: str) -> maix.err.Err
```

Load model from file

item	description
type	func
param	**model**: Model path want to load
return	err::Err
static	False

> C++ defination code:
>
> ```
> err::Err load(const string &model)
> ```

5.14.3. detect

```
def detect(self, img: maix.image.Image, conf_th: float = 0.4, iou_th: float = 0.45, fit: maix.image.Fit = ...) -> list[...]
```

Detect objects from image

item	description
type	func
param	**img**: Image want to detect, if image's size not match model input's, will auto resize with fit method. **conf_th**: Confidence threshold, default 0.4. **iou_th**: IoU threshold, default 0.45. **fit**: Resize method, default image.Fit.FIT_CONTAIN.
throw	If image format not match model input format, will throw err::Exception.
return	Object list. In C++, you should delete it after use.
static	False

> C++ defination code:
>
> ```
> std::vector<nn::Object> *detect(image::Image &img, float conf_th = 0.4, float iou_th = 0.45, maix::image::Fit fit = maix::image::FIT_CONTAIN)
> ```

5.14.4. input_size

```
def input_size(self) -> maix.image.Size
```

Get model input size

item	description
type	func
return	model input size
static	False

> C++ defination code:
>
> ```
> image::Size input_size()
> ```

5.14.5. input_width

```
def input_width(self) -> int
```

Get model input width

item	description
type	func
return	model input size of width
static	False

> C++ defination code:
>
> ```
> int input_width()
> ```

5.14.6. input_height

```
def input_height(self) -> int
```

Get model input height

item	description
type	func
return	model input size of height
static	False

> C++ defination code:
>
> ```
> int input_height()
> ```

5.14.7. input_format

```
def input_format(self) -> maix.image.Format
```

Get input image format

item	description
type	func
return	input image format, image::Format type.
static	False

> C++ defination code:
>
> ```
> image::Format input_format()
> ```

5.14.8. mean

Get mean value, list type

item	description
type	var
static	False
readonly	False

C++ defination code:

```
std::vector<float> mean
```

5.14.9. scale

Get scale value, list type

item	description
type	var
static	False
readonly	False

C++ defination code:

```
std::vector<float> scale
```

5.15. Object

Object for detect result

C++ defination code:

```
class Object
```

5.15.1. __init__

```python
def __init__(self, x: int = 0, y: int = 0, w: int = 0, h: int = 0, class_id: int = 0, score: float = 0, points: list[int] = [], angle: float = -9999) -> None
```

Constructor of Object for detect result

item	description
type	func
param	**x**: left top x **y**: left top y **w**: width **h**: height **class_id**: class id **score**: score
static	False

> C++ defination code:
>
> ```cpp
> Object(int x = 0, int y = 0, int w = 0, int h = 0, int class_id = 0, float score = 0, std::vector<int> points = std::vector<int>(), float angle = -9999)
> ```

5.15.2. __str__

```python
def __str__(self) -> str
```

Object info to string

item	description
type	func
return	Object info string
static	False

> C++ defination code:
>
> ```
> std::string to_str()
> ```

5.15.3. get_obb_points

```
def get_obb_points(self) ->
list[int]
```

Get OBB(oriented bounding box) points, auto calculated according to x,y,w,h,angle

item	description
type	func
static	False

> C++ defination code:
>
> ```
> std::vector<int> get_obb_points()
> ```

5.15.4. x

Object left top coordinate x

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> int x
> ```

5.15.5. y

Object left top coordinate y

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> int y
> ```

5.15.6. w

Object width

item	description
type	var
static	False

item	description
readonly	False

C++ defination code:

```
int w
```

5.15.7. h

Object height

item	description
type	var
static	False
readonly	False

C++ defination code:

```
int h
```

5.15.8. class_id

Object class id

item	description
type	var
static	False
readonly	False

C++ defination code:

```
int class_id
```

5.15.9. score

Object score

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> float score
> ```

5.15.10. points

keypoints

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<int> points
> ```

5.15.11. angle

Rotate angle, -9999 means not set, value is a percentage, need to multiply 180 to get the real angle or multiply PI to get the radian.

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> float angle
> ```

5.15.12. seg_mask

segmentation mask, uint8 list type, shape is h * w but flattened to one dimension, value fron 0 to 255.

item	description
type	var
attention	For efficiency, it's a pointer in C++, use this carefully!
static	False
readonly	False

> C++ defination code:
>
> ```
> image::Image *seg_mask
> ```

5.16. ObjectFloat

Object for detect result

> C++ defination code:
>
> ```
> class ObjectFloat
> ```

5.16.1. __init__

```
def __init__(self, x: float = 0, y: float = 0, w: float = 0, h: float = 0, class_id: float = 0, score: float = 0, points: list[float] = [], angle: float = -1) -> None
```

Constructor of Object for detect result

item	description
type	func
param	**x**: left top x **y**: left top y **w**: width **h**: height **class_id**: class id **score**: score
static	False

> C++ defination code:
>
> ```
> ObjectFloat(float x = 0, float y = 0, float w = 0, float h = 0, float class_id = 0, float score =
> ```

```
0, std::vector<float> points =
std::vector<float>(), float angle
= -1)
```

5.16.2. __str__

```
def __str__(self) -> str
```

Object info to string

item	description
type	func
return	Object info string
static	False

> C++ defination code:
>
> ```
> std::string to_str()
> ```

5.16.3. x

Object left top coordinate x

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> float x
> ```

5.16.4. y

Object left top coordinate y

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> float y
> ```

5.16.5. w

Object width

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> float w
> ```

5.16.6. h

Object height

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> float h
> ```

5.16.7. class_id

Object class id

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> float class_id
> ```

5.16.8. score

Object score

item	description
type	var
static	False

item	description
readonly	False

> C++ defination code:
>
> ```
> float score
> ```

5.16.9. points

keypoints

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> points
> ```

5.16.10. angle

Rotate angle

item	description
type	var
static	False
readonly	False

> C++ defination code:

```
float angle
```

5.17. Objects

Objects Class for detect result

> C++ defination code:
>
> ```
> class Objects
> ```

5.17.1. __init__

```
def __init__(self) -> None
```

Constructor of Objects class

item	description
type	func
static	False

> C++ defination code:
>
> ```
> Objects()
> ```

5.17.2. add

```
def add(self, x: int = 0, y: int = 0, w: int = 0, h: int = 0, class_id: int = 0, score: float = 0, points: list[int] = [], angle: float = -1) -> Object
```

Add object to objects

item	description
type	func
throw	Throw exception if no memory
static	False

> C++ defination code:
>
> ```
> nn::Object &add(int x = 0, int y = 0, int w = 0, int h = 0, int class_id = 0, float score = 0, std::vector<int> points = std::vector<int>(), float angle = -1)
> ```

5.17.3. remove

```
def remove(self, idx: int) -> maix.err.Err
```

Remove object form objects

item	description
type	func
static	False

> C++ defination code:
>
> ```
> err::Err remove(int idx)
> ```

5.17.4. at

```
def at(self, idx: int) -> Object
```

Get object item

item	description
type	func
static	False

> C++ defination code:
>
> ```
> nn::Object &at(int idx)
> ```

5.17.5. __getitem__

```
def __getitem__(self, idx: int) ->
Object
```

Get object item

item	description
type	func
static	False

> C++ defination code:
>
> ```
> nn::Object &operator[](int idx)
> ```

5.17.6. __len__

```
def __len__(self) -> int
```

Get size

item	description
type	func
static	False

> C++ defination code:
>
> ```
> size_t size()
> ```

5.17.7. __iter__

```
def __iter__(self) -> typing.Iterator
```

Begin

item	description
type	func
static	False

> C++ defination code:
>
> ```
> std::vector<Object*>::iterator begin()
> ```

5.18. HandLandmarks

HandLandmarks class

> C++ defination code:

```
class HandLandmarks
```

5.18.1. __init__

```
def __init__(self, model: str = '')
-> None
```

Constructor of HandLandmarks class

item	description
type	func
param	**model**: model path, default empty, you can load model later by load function.
throw	If model arg is not empty and load failed, will throw err::Exception.
static	False

> C++ defination code:
>
> ```
> HandLandmarks(const string &model
> = "")
> ```

5.18.2. load

```
def load(self, model: str) ->
maix.err.Err
```

Load model from file

item	description
type	func
param	**model**: Model path want to load
return	err::Err
static	False

> C++ defination code:
>
> ```
> err::Err load(const string &model)
> ```

5.18.3. detect

```
def detect(self, img: maix.image.Image, conf_th: float = 0.7, iou_th: float = 0.45, conf_th2: float = 0.8, landmarks_rel: bool = False) -> Objects
```

Detect objects from image

item	description
type	func
param	**img**: Image want to detect, if image's si model input's, will auto resize with fit m **conf_th**: Hand detect confidence thresh **iou_th**: IoU threshold, default 0.45. **conf_th2**: Hand detect confidence secor threshold, default 0.8. **landmarks_rel**: outputs the relative coo points with respect to the top-left verte:

item	description
	In obj.points, the last 21x2 values are ar x0y0x1y1...x20y20. Value from 0 to obj.w.
throw	If image format not match model input throw err::Exception.
return	Object list. In C++, you should delete it Object's points value format: box_toplef box_topleft_y, box_topright_x, box_topri box_bottomright_x, box_bottomright_y, box_bottomleft_x, box_bottomleft_y, x0, y0, z1, x1, y1, z2, ..., x20, y20, z20. If landmarks_rel is True, will be box_topl box_topleft_y...,x20,y20,z20,x0_rel,y0_rel,. Z is depth, the larger the value, the fart the palm, and the positive value means camera.
static	False

> C++ defination code:
>
> ```
> nn::Objects *detect(image::Image &img, float conf_th = 0.7, float iou_th = 0.45, float conf_th2 = 0.8, bool landmarks_rel = false)
> ```

5.18.4. input_size

```
def input_size(self, detect: bool = True) -> maix.image.Size
```

Get model input size

item	description
type	func
param	**detect**: detect or landmarks model, default true.
return	model input size
static	False

> C++ defination code:
>
> ```
> image::Size input_size(bool detect = true)
> ```

5.18.5. input_width

```
def input_width(self, detect: bool = True) -> int
```

Get model input width

item	description
type	func
param	**detect**: detect or landmarks model, default true.
return	model input size of width
static	False

> C++ defination code:

```
int input_width(bool detect = true)
```

5.18.6. input_height

```
def input_height(self, detect: bool = True) -> int
```

Get model input height

item	description
type	func
param	**detect**: detect or landmarks model, default true.
return	model input size of height
static	False

> C++ defination code:
>
> ```
> int input_height(bool detect = true)
> ```

5.18.7. input_format

```
def input_format(self) -> maix.image.Format
```

Get input image format

item	description
type	func
return	input image format, image::Format type.
static	False

> C++ defination code:
>
> ```
> image::Format input_format()
> ```

5.18.8. draw_hand

```
def draw_hand(self, img: maix.image.Image, leftright: int, points: list[int], r_min: int = 4, r_max: int = 10, box: bool = True, box_thickness: int = 1, box_color_l: maix.image.Color = ..., box_color_r: maix.image.Color = ...) -> None
```

Draw hand and landmarks on image

item	description
type	func
param	**img**: image object, maix.image.Image type. **leftright,**: 0 means left, 1 means right **points**: points result from detect method: box_topleft_x, box_topleft_y, box_topright_x, box_topright_y, box_bottomright_x,

item	description
	box_bottomright_y, box_bottomleft_x, box_bottomleft_y, x0, y0, z1, x1, y1, z2, ..., x20, y20, z20 **r_min**: min radius of points. **r_max**: min radius of points. **box**: draw box or not, default true. **box_color**: color of box.
static	False

> C++ defination code:
>
> ```
> void draw_hand(image::Image &img, int leftright, const std::vector<int> &points, int r_min = 4, int r_max = 10, bool box = true, int box_thickness = 1, image::Color box_color_l = image::COLOR_RED, image::Color box_color_r = image::COLOR_GREEN)
> ```

5.18.9. labels

Labels list

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<string> labels
> ```

5.18.10. label_path

Label file path

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::string label_path
> ```

5.18.11. mean

Get mean value, list type

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> mean
> ```

5.18.12. scale

Get scale value, list type

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> scale
> ```

5.19. Speech

Speech

> C++ defination code:
>
> ```
> class Speech
> ```

5.19.1. __init__

```
def __init__(self, model: str = '')
-> None
```

Construct a new Speech object

item	description
type	func
param	**model**: model path, default empty, you can load model later by load function.

item	description
throw	If model arg is not empty and load failed, will throw err::Exception.
static	False

> C++ defination code:
>
> ```
> Speech(const string &model = "")
> ```

5.19.2. load

```
def load(self, model: str) ->
maix.err.Err
```

Load model from file

item	description
type	func
param	**model**: Model path want to load
return	err::Err
static	False

> C++ defination code:
>
> ```
> err::Err load(const string
> &model)
> ```

5.19.3. init

```
def init(self, dev_type: SpeechDevice, device_name: str = '') -> maix.err.Err
```

Init the ASR library and select the type and name of the audio device.

item	description
type	func
param	**dev_type**: device type want to detect, can choose between WAV, PCM, or MIC. **device_name**: device name want to detect, can choose a WAV file, a PCM file, or a MIC device name.
throw	**1**. If am model is not loaded, will throw err::ERR_NOT_IMPL. **2**. If device is not supported, will throw err::ERR_NOT_IMPL.
return	err::Err type, if init success, return err::ERR_NONE
static	False

> C++ defination code:
>
> ```
> err::Err init(nn::SpeechDevice dev_type, const string &device_name = "")
> ```

5.19.4. devive

```python
def devive(self, dev_type: SpeechDevice, device_name: str) -> maix.err.Err
```

Reset the device, usually used for PCM/WAV recognition,\nsuch as identifying the next WAV file.

item	description
type	func
param	**dev_type**: device type want to detect, can choose between WAV, PCM, or MIC. **device_name**: device name want to detect, can choose a WAV file, a PCM file, or a MIC device name.
throw	If device is not supported, will throw err::ERR_NOT_IMPL.
return	err::Err type, if init success, return err::ERR_NONE
static	False

> C++ defination code:
>
> ```cpp
> err::Err devive(nn::SpeechDevice dev_type, const string &device_name)
> ```

5.19.5. dec_deinit

```python
def dec_deinit(self, decoder: SpeechDecoder) -> None
```

Deinit the decoder.

item	description
type	func
param	**decoder**: decoder type want to deinit can choose between DECODER_RAW, DECODER_DIG, DECODER_LVCSR, DECODER_KWS or DECODER_ALL.
throw	If device is not supported, will throw err::ERR_NOT_IMPL.
static	False

> C++ defination code:
>
> ```
> void dec_deinit(nn::SpeechDecoder decoder)
> ```

5.19.6. raw

```
def raw(self, callback: typing.Callable[[list[tuple[int, float]], int], None]) -> maix.err.Err
```

Init raw decoder, it will output the prediction results of the original AM.

item	description
type	func
param	**callback**: raw decoder user callback.

item	description
return	err::Err type, if init success, return err::ERR_NONE
static	False

> C++ defination code:
>
> ```
> err::Err raw(std::function<void(std::vector<s
> float>>, int)> callback)
> ```

5.19.7. digit

```
def digit(self, blank: int, callback: typing.Callable[[str, int], None]) -> maix.err.Err
```

Init digit decoder, it will output the Chinese digit recognition results within the last 4 seconds.

item	description
type	func
param	**blank**: If it exceeds this value, insert a '_' in the output result to indicate idle mute. **callback**: digit decoder user callback.
return	err::Err type, if init success, return err::ERR_NONE
static	False

> C++ defination code:
>
> ```
> err::Err digit(int blank, std::function<void(char*, int)> callback)
> ```

5.19.8. kws

```
def kws(self, kw_tbl: list[str], kw_gate: list[float], callback: typing.Callable[[list[float], int], None], auto_similar: bool = True) -> maix.err.Err
```

Init kws decoder, it will output a probability list of all registered keywords in the latest frame,\nusers can set their own thresholds for wake-up.

item	description
type	func
param	**kw_tbl**: Keyword list, filled in with spaces separated by pinyin, for example: xiao3 ai4 tong2 xue2 **kw_gate**: kw_gate, keyword probability gate table, the number should be the same as kw_tbl **auto_similar**: Whether to perform automatic homophone processing, setting it to true will automatically calculate the probability by using pinyin with different tones as

item	description
	homophones **callback**: digit decoder user callback.
return	err::Err type, if init success, return err::ERR_NONE
static	False

> C++ defination code:
>
> ```
> err::Err kws(std::vector<string> kw_tbl, std::vector<float> kw_gate, std::function<void(std::vector<float
> int)> callback, bool auto_similar = true)
> ```

5.19.9. lvcsr

```
def lvcsr(self, sfst_name: str, sym_name: str, phones_txt: str, words_txt: str, callback: typing.Callable[[tuple[str, str], int], None], beam: float = 8, bg_prob: float = 10, scale: float = 0.5, mmap: bool = False) -> maix.err.Err
```

Init lvcsr decoder, it will output continuous speech recognition results (less than 1024 Chinese characters).

item	description
type	func

item	description
param	**sfst_name**: Sfst file path. **sym_name**: Sym file path (output symbol table). **phones_txt**: Path to phones.bin (pinyin table). **words_txt**: Path to words.bin (dictionary table). **callback**: lvcsr decoder user callback. **beam**: The beam size for WFST search is set to 8 by default, and it is recommended to be between 3 and 9. The larger the size, the larger the search space, and the more accurate but slower the search. **bg_prob**: The absolute value of the natural logarithm of the default probability value for background pinyin outside of BEAM-CNT is set to 10 by default. **scale**: acoustics_cost = log(pny_prob)*scale.* *mmap**: use mmap to load the WFST decoding image, If set to true, the beam should be less than 5.
return	err::Err type, if init success, return err::ERR_NONE

item	description
static	False

> C++ defination code:
>
> ```
> err::Err lvcsr(const string &sfst_name, const string &sym_name,
>                              const string &phones_txt, const string &words_txt,
>                              std::function<void(std::pair<char*, char*>, int)> callback,
>                              float beam = 8, float bg_prob = 10, float scale = 0.5, bool mmap = false)
> ```

5.19.10. run

```
def run(self, frame: int) -> int
```

Run speech recognition, user can run 1 frame at a time and do other processing after running,\nor it can run continuously within a thread and be stopped by an external thread.

item	description
type	func
param	**frame**: The number of frames per run.

item	description
return	int type, return actual number of frames in the run.
static	False

> C++ defination code:
>
> ```
> int run(int frame)
> ```

5.19.11. clear

```
def clear(self) -> None
```

Reset internal cache operation

item	description
type	func
static	False

> C++ defination code:
>
> ```
> void clear()
> ```

5.19.12. frame_time

```
def frame_time(self) -> int
```

Get the time of one frame.

item	description
type	func
return	int type, return the time of one frame.
static	False

> C++ defination code:
>
> ```
> int frame_time()
> ```

5.19.13. similar

```
def similar(self, pny: str,
similar_pnys: list[str]) ->
maix.err.Err
```

Manually register mute words, and each pinyin can register up to 10 homophones,\nplease note that using this interface to register homophones will overwrite,\nthe homophone table automatically generated in the "automatic homophone processing" feature.

item	description
type	func
param	**dev_type**: device type want to detect, can choose between WAV, PCM, or MIC. **device_name**: device name want to

item	description
	detect, can choose a WAV file, a PCM file, or a MIC device name.
return	err::Err type, if init success, return err::ERR_NONE
static	False

> C++ defination code:
>
> ```
> err::Err similar(const string &pny, std::vector<std::string> similar_pnys)
> ```

5.19.14. skip_frames

```
def skip_frames(self, num: int) -> None
```

Run some frames and drop, this can be used to avoid\nincorrect recognition results when switching decoders.

item	description
type	func
param	**num**: number of frames to run and drop
static	False

> C++ defination code:
>
> ```
> void skip_frames(int num)
> ```

5.19.15. mean

Get mean value, list type

item	description
type	var
static	False
readonly	False

C++ defination code:

```
std::vector<float> mean
```

5.19.16. scale

Get scale value, list type

item	description
type	var
static	False
readonly	False

C++ defination code:

```
std::vector<float> scale
```

5.19.17. dev_type

```
def dev_type(self) -> SpeechDevice
```

get device type

item	description
type	func
return	nn::SpeechDevice type, see SpeechDevice of this module
static	False

> C++ defination code:
>
> ```
> nn::SpeechDevice dev_type()
> ```

5.20. Classifier

Classifier

> C++ defination code:
>
> ```
> class Classifier
> ```

5.20.1. __init__

```
def __init__(self, model: str = '', dual_buff: bool = True) -> None
```

Construct a new Classifier object

item	description
type	func

item	description
param	**model**: MUD model path, if empty, will not load model, you can call load() later. if not empty, will load model and will raise err::Exception if load failed. **dual_buff**: direction [in], prepare dual input output buffer to accelarate forward, that is, when NPU is forwarding we not wait and prepare the next input buff. If you want to ensure every time forward output the input's result, set this arg to false please. Default true to ensure speed.
static	False

> C++ defination code:
>
> ```
> Classifier(const string &model = "", bool dual_buff = true)
> ```

5.20.2. load

```
def load(self, model: str) -> maix.err.Err
```

Load model from file, model format is .mud,\nMUD file should contain [extra] section, have key-values:\n- model_type: classifier\n- input_type: rgb or bgr\n- mean: 123.675, 116.28, 103.53\n- scale:

0.017124753831663668, 0.01750700280112045, 0.017429193899782137\n- labels: imagenet_classes.txt

item	description
type	func
param	**model**: MUD model path
return	error code, if load failed, return error code
static	False

> C++ defination code:
>
> ```
> err::Err load(const string &model)
> ```

5.20.3. classify

```
def classify(self, img: maix.image.Image, softmax: bool = True, fit: maix.image.Fit = ...) -> list[tuple[int, float]]
```

Forward image to model, get result. Only for image input, use classify_raw for tensor input.

item	description
type	func
param	**img**: image, format should match model input_type， or will raise err.Exception

item	description
	softmax: if true, will do softmax to result, or will return raw value **fit**: image resize fit mode, default Fit.FIT_COVER, see image.Fit.
throw	If error occurred, will raise err::Exception, you can find reason in log, mostly caused by args error or hardware error.
return	result, a list of (label, score). If in dual_buff mode, value can be one element list and score is zero when not ready. In C++, you need to delete it after use.
static	False

> C++ defination code:
>
> ```
> std::vector<std::pair<int, float>> *classify(image::Image &img, bool softmax = true, image::Fit fit = image::FIT_COVER)
> ```

5.20.4. classify_raw

```
def classify_raw(self, data: maix.tensor.Tensor, softmax: bool = True) -> list[tuple[int, float]]
```

Forward tensor data to model, get result

item	description
type	func
param	**data**: tensor data, format should match model input_type， or will raise err.Excetion **softmax**: if true, will do softmax to result, or will return raw value
throw	If error occurred, will raise err::Exception, you can find reason in log, mostly caused by args error or hardware error.
return	result, a list of (label, score). In C++, you need to delete it after use.
static	False

> C++ defination code:
>
> ```
> std::vector<std::pair<int, float>> *classify_raw(tensor::Tensor &data, bool softmax = true)
> ```

5.20.5. input_size

```
def input_size(self) -> maix.image.Size
```

Get model input size, only for image input

item	description
type	func
return	model input size
static	False

> C++ defination code:
>
> ```
> image::Size input_size()
> ```

5.20.6. input_width

```
def input_width(self) -> int
```

Get model input width, only for image input

item	description
type	func
return	model input size of width
static	False

> C++ defination code:
>
> ```
> int input_width()
> ```

5.20.7. input_height

```
def input_height(self) -> int
```

Get model input height, only for image input

item	description
type	func
return	model input size of height
static	False

> C++ defination code:
>
> ```
> int input_height()
> ```

5.20.8. input_format

```
def input_format(self) ->
maix.image.Format
```

Get input image format, only for image input

item	description
type	func
return	input image format, image::Format type.
static	False

> C++ defination code:
>
> ```
> image::Format input_format()
> ```

5.20.9. input_shape

```
def input_shape(self) -> list[int]
```

Get input shape, if have multiple input, only return first input shape

item	description
type	func
return	input shape, list type
static	False

> C++ defination code:
>
> ```
> std::vector<int> input_shape()
> ```

5.20.10. labels

Labels list

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<string> labels
> ```

5.20.11. label_path

Label file path

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::string label_path
> ```

5.20.12. mean

Get mean value, list type

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> mean
> ```

5.20.13. scale

Get scale value, list type

item	description
type	var
static	False

item	description
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> scale
> ```

5.21. PP_OCR

PP_OCR class

> C++ defination code:
>
> ```
> class PP_OCR
> ```

5.21.1. __init__

```
def __init__(self, model: str = '')
    -> None
```

Constructor of PP_OCR class

item	description
type	func
param	**model**: model path, default empty, you can load model later by load function.
throw	If model arg is not empty and load failed, will throw err::Exception.
static	False

> C++ defination code:
>
> ```
> PP_OCR(const string &model = "")
> ```

5.21.2. load

```
def load(self, model: str) -> maix.err.Err
```

Load model from file

item	description
type	func
param	**model**: Model path want to load
return	err::Err
static	False

> C++ defination code:
>
> ```
> err::Err load(const string &model)
> ```

5.21.3. detect

```
def detect(self, img: maix.image.Image, thresh: float = 0.3, box_thresh: float = 0.6, fit: maix.image.Fit = ..., char_box: bool = False) -> OCR_Objects
```

Detect objects from image

item	description
type	func
param	**img**: Image want to detect, if image's size not match model input's, will auto resize with fit method. **thresh**: Confidence threshold where pixels have charactor, default 0.3. **box_thresh**: Box threshold, the box prob higher than this value will be valid, default 0.6. **fit**: Resize method, default image.Fit.FIT_CONTAIN. **char_box**: Calculate every charactor's box, default false, if true then you can get charactor's box by nn.OCR_Object's char_boxes attribute.
throw	If image format not match model input format or no memory, will throw err::Exception.
return	nn.OCR_Objects type. In C++, you should delete it after use.
static	False

> C++ defination code:
>
> ```
> nn::OCR_Objects *detect(image::Image &img, float thresh = 0.3, float box_thresh = 0.6, maix::image::Fit fit = maix::image::FIT_CONTAIN, bool char_box = false)
> ```

5.21.4. recognize

```python
def recognize(self, img: maix.image.Image, box_points: list[int] = []) -> OCR_Object
```

Only recognize, not detect

item	description
type	func
param	**img**: image to recognize chractors, can be a stanrd cropped charactors image, if crop image not standard, you can use box_points to assgin where the charactors' 4 corner is. **box_points**: list type, length must be 8 or 0, default empty means not transfer image to standard image. 4 points postiion, format: [x1, y1, x2, y2, x3, y3, x4, y4], point 1 at the left-top, point 2 right-top... **char_box**: Calculate every charactor's box, default false, if true then you can get charactor's box by nn.OCR_Object's char_boxes attribute.
static	False

> C++ defination code:
>
> ```cpp
> nn::OCR_Object *recognize(image::Image &img, const std::vector<int>
> ```

```
&box_points = std::vector<int>())
```

5.21.5. draw_seg_mask

```
def draw_seg_mask(self, img: maix.image.Image, x: int, y: int, seg_mask: maix.image.Image, threshold: int = 127) -> None
```

Draw segmentation on image

item	description
type	func
param	**img**: image object, maix.image.Image type. **seg_mask**: segmentation mask image by detect method, a grayscale image **threshold**: only mask's value > threshold will be draw on image, value from 0 to 255.
static	False

> C++ defination code:
>
> ```
> void draw_seg_mask(image::Image &img, int x, int y, image::Image &seg_mask, int threshold = 127)
> ```

5.21.6. input_size

```
def input_size(self) -> maix.image.Size
```

Get model input size

item	description
type	func
return	model input size
static	False

> C++ defination code:
>
> ```
> image::Size input_size()
> ```

5.21.7. input_width

```
def input_width(self) -> int
```

Get model input width

item	description
type	func
return	model input size of width
static	False

> C++ defination code:
>
> ```
> int input_width()
> ```

5.21.8. input_height

```
def input_height(self) -> int
```

Get model input height

item	description
type	func
return	model input size of height
static	False

> C++ defination code:
>
> ```
> int input_height()
> ```

5.21.9. input_format

```
def input_format(self) -> maix.image.Format
```

Get input image format

item	description
type	func
return	input image format, image::Format type.
static	False

> C++ defination code:
>
> ```
> image::Format input_format()
> ```

5.21.10. mean

Get mean value, list type

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> mean
> ```

5.21.11. scale

Get scale value, list type

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> scale
> ```

5.21.12. rec_mean

Get mean value, list type

item	description
type	var

item	description
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> rec_mean
> ```

5.21.13. rec_scale

Get scale value, list type

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> rec_scale
> ```

5.21.14. labels

labels (charactors)

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<std::string> labels
> ```

5.21.15. det

model have detect model

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> bool det
> ```

5.21.16. rec

model have recognize model

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> bool rec
> ```

5.22. YOLOv5

YOLOv5 class

> C++ defination code:
>
> ```
> class YOLOv5
> ```

5.22.1. __init__

```
def __init__(self, model: str = '',
dual_buff: bool = True) -> None
```

Constructor of YOLOv5 class

item	description
type	func
param	**model**: model path, default empty, you can load model later by load function. **dual_buff**: direction [in], prepare dual input output buffer to accelarate forward, that is, when NPU is forwarding we not wait and prepare the next input buff. If you want to ensure every time forward output the input's result, set this arg to false please. Default true to ensure speed.
throw	If model arg is not empty and load failed, will throw err::Exception.

item	description
static	False

> C++ defination code:
>
> ```
> YOLOv5(const string &model = "", bool dual_buff = true)
> ```

5.22.2. load

```
def load(self, model: str) -> maix.err.Err
```

Load model from file

item	description
type	func
param	**model**: Model path want to load
return	err::Err
static	False

> C++ defination code:
>
> ```
> err::Err load(const string &model)
> ```

5.22.3. detect

```
def detect(self, img: maix.image.Image, conf_th: float = 0.5, iou_th: float = 0.45, fit:
```

```
    maix.image.Fit = ..., sort: int = 0)
    -> list[Object]
```

Detect objects from image

item	description
type	func
param	**img**: Image want to detect, if image's size not match model input's, will auto resize with fit method. **conf_th**: Confidence threshold, default 0.5. **iou_th**: IoU threshold, default 0.45. **fit**: Resize method, default image.Fit.FIT_CONTAIN. **sort**: sort result according to object size, default 0 means not sort, 1 means bigger in front, -1 means smaller in front.
throw	If image format not match model input format, will throw err::Exception.
return	Object list. In C++, you should delete it after use.
static	False

> C++ defination code:
>
> ```
> std::vector<nn::Object>
> *detect(image::Image &img, float
> conf_th = 0.5, float iou_th =
> 0.45, maix::image::Fit fit =
> ```

```
maix::image::FIT_CONTAIN, int
sort = 0)
```

5.22.4. input_size

```
def input_size(self) ->
maix.image.Size
```

Get model input size

item	description
type	func
return	model input size
static	False

> C++ defination code:
>
> ```
> image::Size input_size()
> ```

5.22.5. input_width

```
def input_width(self) -> int
```

Get model input width

item	description
type	func
return	model input size of width
static	False

> C++ defination code:
>
> ```
> int input_width()
> ```

5.22.6. input_height

```
def input_height(self) -> int
```

Get model input height

item	description
type	func
return	model input size of height
static	False

> C++ defination code:
>
> ```
> int input_height()
> ```

5.22.7. input_format

```
def input_format(self) ->
maix.image.Format
```

Get input image format

item	description
type	func

item	description
return	input image format, image::Format type.
static	False

> C++ defination code:
>
> ```
> image::Format input_format()
> ```

5.22.8. labels

Labels list

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<string> labels
> ```

5.22.9. label_path

Label file path

item	description
type	var
static	False

item	description
readonly	False

> C++ defination code:
>
> ```
> std::string label_path
> ```

5.22.10. mean

Get mean value, list type

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> mean
> ```

5.22.11. scale

Get scale value, list type

item	description
type	var
static	False
readonly	False

> C++ defination code:

```
std::vector<float> scale
```

5.22.12. anchors

Get anchors

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<float> anchors
> ```

5.23. MUD

MUD(model universal describe file) class

> C++ defination code:
>
> ```
> class MUD
> ```

5.23.1. __init__

```
def __init__(self, model_path: str = '') -> None
```

MUD constructor

item	description
type	func
param	**model_path**: direction [in], model file path, model format can be MUD(model universal describe file) file. If model_path set, will load model from file, load failed will raise err.Exception. If model_path not set, you can load model later by load function.
static	False

> C++ defination code:
>
> ```
> MUD(const std::string &model_path = "")
> ```

5.23.2. load

```
def load(self, model_path: str) -> maix.err.Err
```

Load model from file

item	description
type	func
param	**model_path**: direction [in], model file path, model format can be

item	description
	MUD(model universal describe file) file.
return	error code, if load success, return err::ERR_NONE
static	False

> C++ defination code:
>
> ```
> err::Err load(const std::string &model_path)
> ```

5.23.3. type

Model type, string type

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::string type
> ```

5.23.4. items

Model config items, different model type has different config items

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::map<std::string, std::map<std::string, std::string>> items
> ```

5.23.5. model_path

Model path

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::string model_path
> ```

5.23.6. parse_labels

```
def parse_labels(self, key: str = 'labels') -> list[str]
```

Please load() first, parse labels in items["extra"]["labels"],\nif items["extra"]["labels"] is a file path: will parse file, every one line is a label;\nif items["extra"]["labels"] is a string, labels split by comma(",").\nExecute this method will replace items["extra"]["labels"];

item	description
type	func
param	**key**: parse from items[key], default "labels".
return	parsed labels list.
static	False

> C++ defination code:
>
> ```
> std::vector<std::string> parse_labels(const std::string key = "labels")
> ```

5.24. LayerInfo

NN model layer info

> C++ defination code:
>
> ```
> class LayerInfo
> ```

5.24.1. __init__

```python
def __init__(self, name: str = '', dtype: maix.tensor.DType = ..., shape: list[int] = []) -> None
```

LayerInfo constructor

item	description
type	func
param	**name**: direction [in], layer name **dtype**: direction [in], layer data type **shape**: direction [in], layer shape
static	False

> C++ defination code:
>
> ```cpp
> LayerInfo(const std::string &name = "", tensor::DType dtype = tensor::DType::FLOAT32, std::vector<int> shape = std::vector<int>())
> ```

5.24.2. name

Layer name

item	description
type	var
static	False
readonly	False

> C++ defination code:

```
std::string   name
```

5.24.3. dtype

Layer data type

item	description
type	var
attention	If model is quantized, this is the real quantized data type like int8 float16, in most scene, inputs and outputs we actually use float32 in API like forward.
static	False
readonly	False

> C++ defination code:
>
> ```
> tensor::DType dtype
> ```

5.24.4. shape

Layer shape

item	description
type	var
static	False
readonly	False

> C++ defination code:
>
> ```
> std::vector<int> shape
> ```

5.24.5. shape_int

```
def shape_int(self) -> int
```

Shape as one int type, multiply all dims of shape

item	description
type	func
static	False

> C++ defination code:
>
> ```
> int shape_int()
> ```

5.24.6. to_str

```
def to_str(self) -> str
```

To string

item	description
type	func
static	False

> C++ defination code:

```
std::string to_str()
```

5.24.7. __str__

```
def __str__(self) -> str
```

To string

item	description
type	func
static	False

> C++ defination code:
>
> ```
> std::string __str__()
> ```

5.25. NN

Neural network class

> C++ defination code:
>
> ```
> class NN
> ```

5.25.1. __init__

```
def __init__(self, model: str = '', dual_buff: bool = False) -> None
```

Neural network constructor

item	description
type	func
param	**model**: direction [in], model file path, model format can be MUD(model universal describe file) file. If model_path set, will load model from file, load failed will raise err.Exception. If model_path not set, you can load model later by load function. **dual_buff**: direction [in], prepare dual input output buffer to accelarate forward, that is, when NPU is forwarding we not wait and prepare the next input buff. If you want to ensure every time forward output the input's result, set this arg to false please. Default false to ensure easy use.
static	False

> C++ defination code:
>
> ```
> NN(const std::string &model = "", bool dual_buff = false)
> ```

5.25.2. load

```
def load(self, model: str) -> maix.err.Err
```

Load model from file

item	description
type	func
param	**model**: direction [in], model file path, model format can be MUD(model universal describe file) file.
return	error code, if load success, return err::ERR_NONE
static	False

> C++ defination code:
>
> ```
> err::Err load(const std::string &model)
> ```

5.25.3. loaded

```
def loaded(self) -> bool
```

Is model loaded

item	description
type	func
return	true if model loaded, else false
static	False

> C++ defination code:

```
bool loaded()
```

5.25.4. set_dual_buff

```
def set_dual_buff(self, enable: bool) -> None
```

Enable dual buff or disable dual buff

item	description
type	func
param	**enable**: true to enable, false to disable
static	False

> C++ defination code:
>
> ```
> void set_dual_buff(bool enable)
> ```

5.25.5. inputs_info

```
def inputs_info(self) -> list[LayerInfo]
```

Get model input layer info

item	description
type	func
return	input layer info

item	description
static	False

> C++ defination code:
>
> ```
> std::vector<nn::LayerInfo> inputs_info()
> ```

5.25.6. outputs_info

```
def outputs_info(self) -> list[LayerInfo]
```

Get model output layer info

item	description
type	func
return	output layer info
static	False

> C++ defination code:
>
> ```
> std::vector<nn::LayerInfo> outputs_info()
> ```

5.25.7. extra_info

```
def extra_info(self) -> dict[str, str]
```

Get model extra info define in MUD file

item	description
type	func
return	extra info, dict type, key-value object, attention: key and value are all string type.
static	False

> C++ defination code:
>
> ```
> std::map<std::string, std::string> extra_info()
> ```

5.25.8. extra_info_labels

```
def extra_info_labels(self) -> list[str]
```

Get model parsed extra info labels define in MUD file

item	description
type	func
return	labels list in extra info, string list type.
static	False

> C++ defination code:
>
> ```
> std::vector<std::string> extra_info_labels()
> ```

5.25.9. forward

```
def forward(self, inputs: maix.tensor.Tensors, copy_result: bool = True, dual_buff_wait: bool = False) -> maix.tensor.Tensors
```

forward run model, get output of model,\nthis is specially for MaixPy, not efficient, but easy to use in MaixPy

item	description
type	func
param	**input**: direction [in], input tensor **copy_result**: If set true, will copy result to a new variable; else will use a internal memory, you can only use it until to the next forward. Default true to avoid problems, you can set it to false manually to make speed faster. **dual_buff_wait**: bool type, only for dual_buff mode, if true, will inference this image and wait for result, default false.
return	output tensor. In C++, you should manually delete tensors in return value and return value. If dual_buff mode, it can be NULL(None in MaixPy) means not ready.

item	description
throw	if error ocurrs like no memory or arg error, will raise err.Exception.
static	False

> C++ defination code:
>
> ```
> tensor::Tensors *forward(tensor::Tensors &inputs, bool copy_result = true, bool dual_buff_wait = false)
> ```

5.25.10. forward_image

```
def forward_image(self, img: maix.image.Image, mean: list[float] = [], scale: list[float] = [], fit: maix.image.Fit = ..., copy_result: bool = True, dual_buff_wait: bool = False, chw: bool = True) -> maix.tensor.Tensors
```

forward model, param is image

item	description
type	func
param	**img**: input image **mean**: mean value, a list type, e.g. [0.485, 0.456, 0.406], default is empty list means not normalize. **scale**: scale value, a list type, e.g. [1/0.229, 1/0.224, 1/0.225], default is empty list means not normalize.

item	description
	fit: fit mode, if the image size of input not equal to model's input, it will auto resize use this fit method, default is image.Fit.FIT_FILL for easy coordinate calculation, but for more accurate result, use image.Fit.FIT_CONTAIN is better. **copy_result**: If set true, will copy result to a new variable; else will use a internal memory, you can only use it until to the next forward. Default true to avoid problems, you can set it to false manually to make speed faster. **dual_buff_wait**: bool type, only for dual_buff mode, if true, will inference this image and wait for result, default false. **chw**: chw channel format, forward model with hwc format image input if set to false, default true(chw).
return	output tensor. In C++, you should manually delete tensors in return value and return value. If dual_buff mode, it can be NULL(None in MaixPy) means not ready.
throw	If error occurs, like arg error or alloc memory failed, will raise err.Exception.

item	description
static	False

C++ defination code:

```
tensor::Tensors *forward_image(image::Image &img, std::vector<float> mean = std::vector<float>(), std::vector<float> scale = std::vector<float>(), image::Fit fit = image::Fit::FIT_FILL, bool copy_result = true, bool dual_buff_wait = false, bool chw = true)
```